[Technical Engineering Document] Loan Application & Disbursement Aug 24, 2026

Target Release	Aug 26, 2026
Document Status	In progress
Document owner	Haikal Raihansyah

Related Document

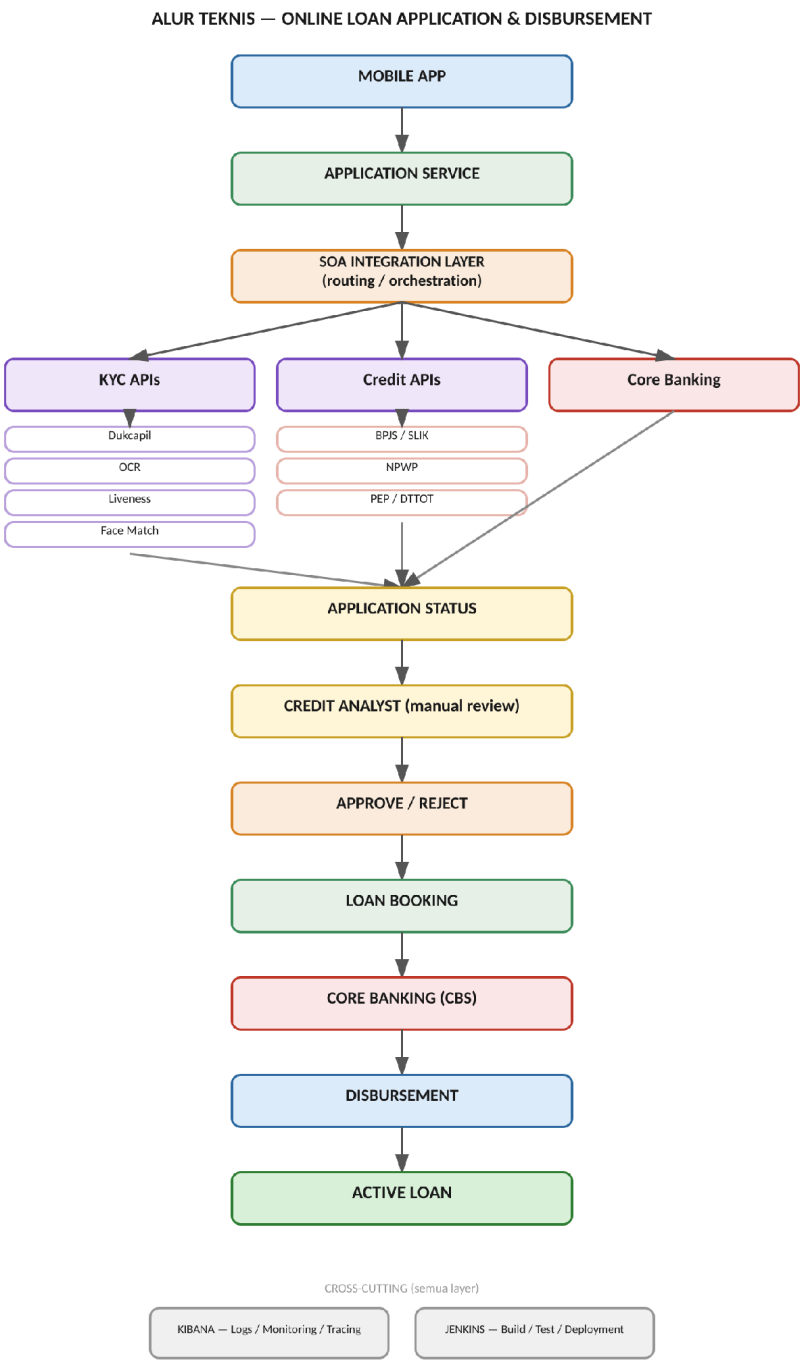
Put the related documents, e.g. docs, sheets, papers, miro, figma, etc.

Document	Link	Notes
Test Technical Analyst	Test Integration Anal...	
[FSD] Loan Application	[FSD] Loan Applicati...	
Flow Diagram	https://whimsical.com/muhammad-haikal-raihansyah/fintech-lending-app-architecture-flow-KXDSHwwLoiuHWdhY2hTaq5	
Github	https://github.com/haikalraihansyah/integration-analyst-test.git	

1. Alur Teknis (Technical Flow)

Dokumen ini menjelaskan alur teknis end-to-end proses pengajuan pinjaman (loan application) hingga pencairan dana (disbursement) secara digital melalui aplikasi mobile. Proses melibatkan orkestrasi terhadap layanan verifikasi identitas (KYC), verifikasi kelayakan kredit, keputusan analis kredit, hingga pencatatan dan pencairan pinjaman di Core Banking System (CBS).

Secara garis besar, permintaan dari Mobile App diteruskan oleh Application Service ke SOA Integration Layer, yang bertugas melakukan routing, orchestration, dan transformation data menuju API eksternal (KYC, credit bureau, compliance) maupun Core Banking. Kibana digunakan untuk logging dan tracing end-to-end, sedangkan Jenkins menangani proses build, test, dan deployment sebagai bagian dari CI/CD pipeline.



Gambar 1. Alur teknis end-to-end — Online Loan Application & Disbursement

1.1 Ringkasan Layer

Layer	Tanggung Jawab
Mobile App	Input data nasabah, upload dokumen, menampilkan status pengajuan.
Application Service	Mengelola business flow, expose API, dan persistensi data pengajuan.
SOA	Routing, orchestration, transformation, dan integrasi ke API eksternal.
External APIs	Verifikasi identitas, employment, credit, dan compliance screening.
Credit Analyst	Keputusan kredit manual dan penentuan limit akhir.

Layer	Tanggung Jawab
Core Banking	Pembukaan rekening pinjaman, booking, dan transaksi finansial.
Kibana	Logging, monitoring, dan distributed tracing.
Jenkins	Build, automated testing, dan deployment (CI/CD).

2. Daftar Core API

Terdapat enam core API utama yang membentuk keseluruhan siklus proses pinjaman, mulai dari verifikasi identitas hingga pencairan dana.

API	Method	Tujuan
/api/v1/kyc/verifications	POST	Menjalankan dan menyimpan hasil KYC.
/api/v1/loan-applications	POST	Membuat pengajuan loan.
/api/v1/loan-applications/{id}/assessment	POST	Menjalankan credit verification.
/api/v1/loan-applications/{id}/decision	POST	Menyimpan keputusan Credit Analyst.
/api/v1/loans	POST	Booking loan ke Core Banking.
/api/v1/disbursements	POST	Memproses pencairan loan.

3. Standar Header API

Seluruh core API menggunakan format header standar berikut agar konsisten dan mendukung end-to-end tracing melalui Kibana.

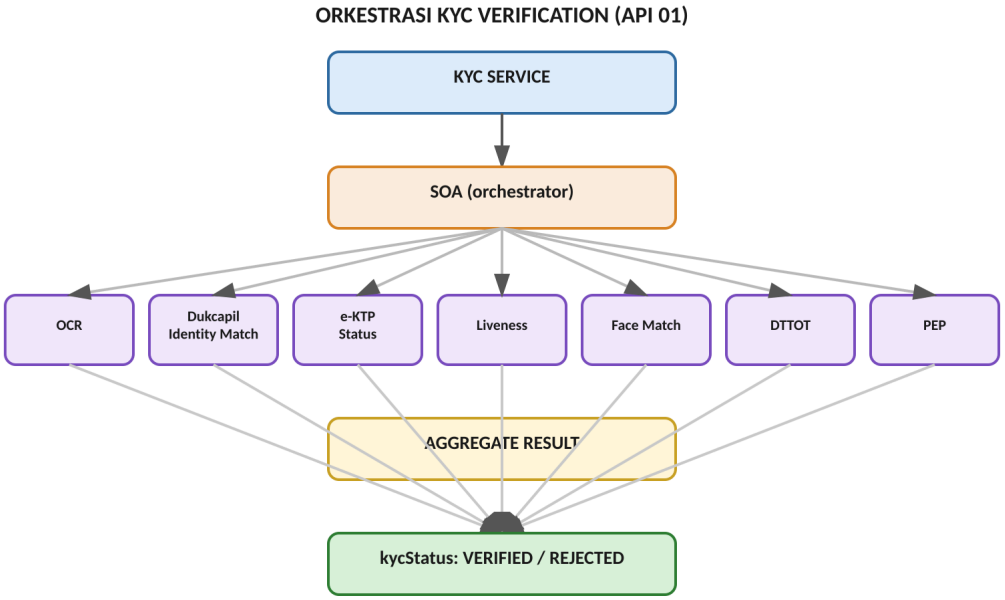
Attribute	Type	Required	Description	Example
Authorization	String	Y	Bearer token	Bearer eyJ...
Content-Type	String	Y	Request format	application/json
X-Correlation-ID	UUID	Y	ID untuk tracing end-to-end	550e8400-e29b...
X-Request-ID	UUID	Y	Unique request ID	REQ-20260826-001
X-Channel	String	Y	Source channel	MOBILE
X-Client-Version	String	N	Mobile version	6.2.1

4. API 01 — KYC Verification

POST /api/v1/kyc/verifications

Purpose

Melakukan verifikasi identitas customer sebelum customer dapat mengajukan loan. Pada API ini, SOA akan mengorkestrasi tujuh layanan verifikasi secara paralel: OCR, Dukcapil Identity Match, e-KTP Status, Liveness, Face Match, DTTOT, dan PEP.



Gambar 2. Orkestrasi KYC Verification oleh SOA

Request Attribute

Attribute	Type	Required	Validation	Example
customerId	String(36)	Y	Existing customer	CUS-000123
documentType	Enum	Y	KTP/PASSPORT	KTP
documentNumber	String(16)	Y	Numeric, 16 digit for KTP	3171234567890001
fullName	String(100)	Y	Minimum 3 characters	Budi Santoso
birthPlace	String(100)	Y	Alphabetic	Jakarta
birthDate	Date	Y	YYYY-MM-DD	1995-08-20
documentImageRef	String(255)	Y	Valid file reference	FILE-123456
faceImageRef	String(255)	Y	Valid file reference	FACE-123456
consentFlag	Boolean	Y	Must be true	true
deviceId	String(100)	Y	Registered device	DEV-12345

Response Attribute

Attribute	Type	Description	Example
kycVerificationId	String(36)	Unique KYC transaction	KYC-000123
customerId	String(36)	Customer reference	CUS-000123
kycStatus	Enum	Overall KYC status	VERIFIED
identityMatchStatus	Enum	Dukcapil matching result	MATCH
identityMatchScore	Decimal(5,4)	Similarity score	0.9985
documentStatus	Enum	KTP status	ACTIVE
livenessStatus	Enum	Liveness result	PASS
faceMatchStatus	Enum	Face matching result	MATCH
faceMatchScore	Decimal(5,4)	Face similarity	0.9721
dttotStatus	Enum	Screening result	CLEAR
pepStatus	Enum	PEP result	CLEAR
verifiedAt	DateTime	Verification timestamp	2026-08-26T10:15:00Z
referenceId	String(50)	Integration reference	KYC-REF-001

Example Response:

01 - KYC Verification — Success


```
{
  "success": true,
  "data": {
    "kycVerificationId": "KYC-000123",
    "customerId": "{{customerId}}",
    "kycStatus": "VERIFIED",
    "identityMatchStatus": "MATCH",
    "identityMatchScore": 0.9985,
    "documentStatus": "ACTIVE",
    "livenessStatus": "PASS",
    "faceMatchStatus": "MATCH",
    "faceMatchScore": 0.9721,
    "dttotStatus": "CLEAR",
    "pepStatus": "CLEAR",
    "verifiedAt": "2026-08-26T10:15:00Z",
    "referenceId": "KYC-REF-001"
  }
}
```


01 - KYC Verification — Rejected (Face Mismatch)


```
{
  "success": true,
  "data": {
    "kycVerificationId": "KYC-000124",
    "customerId": "{{customerId}}",
    "kycStatus": "REJECTED",
    "identityMatchStatus": "MATCH",
    "identityMatchScore": 0.991,
    "documentStatus": "ACTIVE",
    "livenessStatus": "PASS",
    "faceMatchStatus": "NO_MATCH",
    "faceMatchScore": 0.412,
    "dttotStatus": "CLEAR",
    "pepStatus": "CLEAR",
    "verifiedAt": "2026-08-26T10:15:00Z",
    "referenceId": "KYC-REF-002"
  }
}
```

01 - KYC Verification — Validation Error


```
{
  "success": false,
  "error": {
    "code": "KYC-VAL-001",
    "message": "consentFlag must be true",
    "details": {
      "field": "consentFlag",
      "expected": "true",
      "received": false
    },
    "correlationId": "{{correlationId}}",
    "timestamp": "2026-08-26T11:15:00Z"
  }
}
```


5. API 02 — Create Loan Application

POST /api/v1/loan-applications

Purpose

Menyimpan data pengajuan loan setelah customer memiliki KYC yang berstatus VERIFIED.

Request Attribute

Application

Attribute	Type	Required	Validation	Example
customerId	String(36)	Y	Existing customer	CUS-000123
kycVerificationId	String(36)	Y	KYC must be VERIFIED	KYC-000123
loanProductCode	String(20)	Y	Valid product	PERSONAL01
requestedAmount	Decimal(18, 2)	Y	> 0	25000000.00
requestedTenor	Integer	Y	1–60 months	12
loanPurpose	Enum	Y	Valid purpose	PERSONAL
currency	String(3)	Y	ISO currency	IDR

Employment

Attribute	Type	Required	Example
employmentStatus	Enum	Y	EMPLOYEE
employerName	String(150)	Y	PT ABC Indonesia
jobTitle	String(100)	Y	Product Manager
employmentDurationMonth	Integer	Y	48
monthlyIncome	Decimal(18,2)	Y	12000000.00

Financial

Attribute	Type	Required	Example
monthlyExpense	Decimal(18,2)	Y	5000000.00
existingInstallment	Decimal(18,2)	Y	1000000.00
otherIncome	Decimal(18,2)	N	2000000.00
totalMonthlyIncome	Decimal(18,2)	Y	14000000.00

Bank Account

Attribute	Type	Required	Example
bankCode	String(10)	Y	014
accountNumber	String(30)	Y	1234567890
accountName	String(100)	Y	Budi Santoso

Response Attribute

Attribute	Type	Example
applicationId	String(36)	APP-000123
customerId	String(36)	CUS-000123
status	Enum	SUBMITTED
requestedAmount	Decimal	25000000
requestedTenor	Integer	12
createdAt	DateTime	2026-08-26T10:20:00Z
correlationId	UUID	550e8400...

Example Response:

02 - Create Loan Application — Success


```
{
  "applicationId": "APP-000123",
  "customerId": "{{customerId}}",
  "status": "SUBMITTED",
  "requestedAmount": 25000000,
  "requestedTenor": 12,
  "createdAt": "2026-08-26T10:20:00Z",
  "correlationId": "{{correlationId}}"
}
```


02 - Create Loan Application — KYC Not Verified

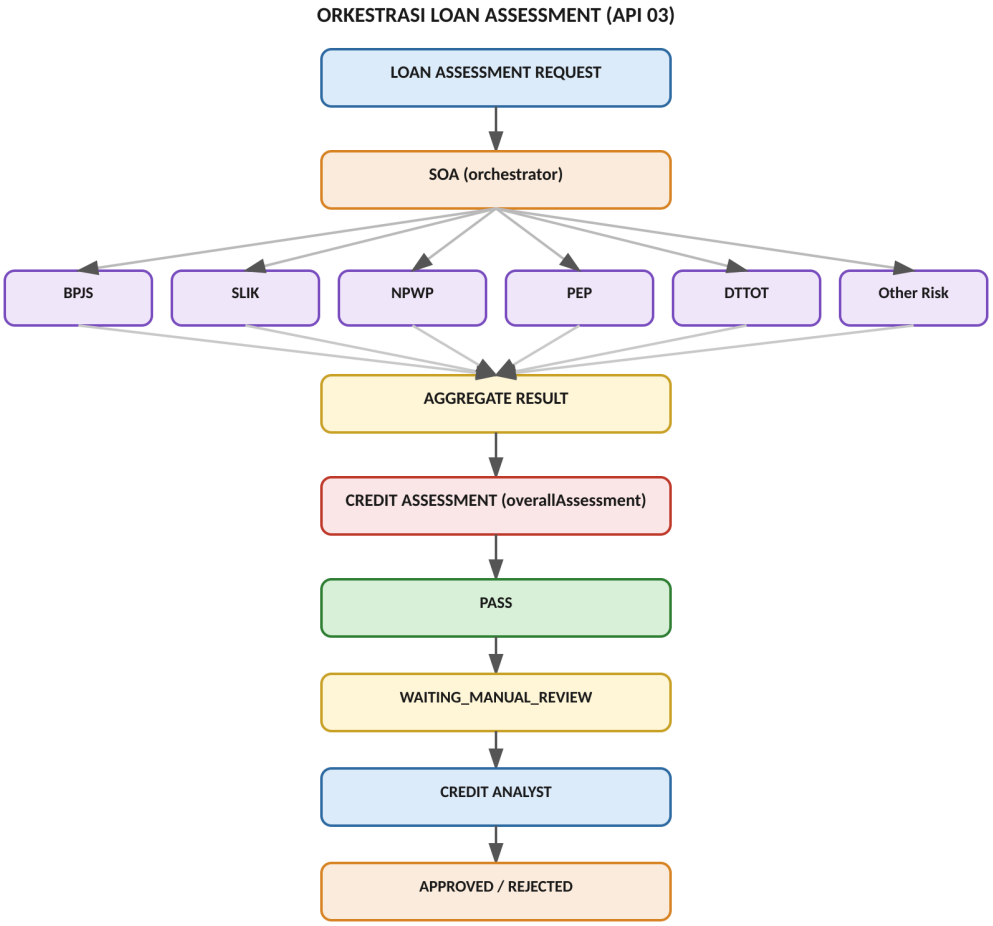

```
{
  "success": false,
  "error": {
    "code": "KYC-VAL-002",
    "message": "KYC status is not VERIFIED",
    "details": {
      "field": "kycVerificationId",
      "expected": "status = VERIFIED",
      "received": "REJECTED"
    },
    "correlationId": "{{correlationId}}",
    "timestamp": "2026-08-26T11:15:00Z"
  }
}
```

6. API 03 — Loan Assessment

POST /api/v1/loan-applications/{applicationId}/assessment

Purpose

Melakukan trigger terhadap SOA untuk menjalankan credit verification terhadap application, mengorkestrasi BPJS, SLIK, NPWP, PEP, DTTOT, dan pengecekan risiko lainnya secara paralel.



Gambar 3. Orkestrasi Loan Assessment oleh SOA

Request Attribute

Attribute	Type	Required	Example
applicationId	String(36)	Y	APP-000123
customerId	String(36)	Y	CUS-000123
assessmentType	Enum	Y	FULL_CREDIT_CHECK
consentId	String(36)	Y	CONSENT-001
requestedAt	DateTime	Y	2026-08-26T10:25:00Z

Response Attribute

Attribute	Type	Example
assessmentId	String(36)	ASM-000123
applicationId	String(36)	APP-000123

Attribute	Type	Example
assessmentStatus	Enum	COMPLETED
bpjsStatus	Enum	VERIFIED
employmentDurationMonth	Integer	48
verifiedMonthlyIncome	Decimal	12000000
slikStatus	Enum	CLEAR
worstCollectibility	Integer	1
totalOutstanding	Decimal	5000000
npwpStatus	Enum	VALID
pepStatus	Enum	CLEAR
dttotStatus	Enum	CLEAR
overallAssessment	Enum	PASS
completedAt	DateTime	2026-08-26T10:30:00Z

Example Response:

03 - Loan Assessment — Success


```
{
  "assessmentId": "ASM-000123",
  "applicationId": "{{applicationId}}",
  "assessmentStatus": "COMPLETED",
  "bpjsStatus": "VERIFIED",
  "employmentDurationMonth": 48,
  "verifiedMonthlyIncome": 12000000,
  "slikStatus": "CLEAR",
  "worstCollectibility": 1,
  "totalOutstanding": 5000000,
  "npwpStatus": "VALID",
  "pepStatus": "CLEAR",
  "dttotStatus": "CLEAR",
  "overallAssessment": "PASS",
  "completedAt": "2026-08-26T10:30:00Z"
}
```


03 - Loan Assessment — SLIK Collectibility Fail


```
{
  "assessmentId": "ASM-000123",
  "applicationId": "{{applicationId}}",
  "assessmentStatus": "COMPLETED",
  "bpjsStatus": "VERIFIED",
  "employmentDurationMonth": 48,
  "verifiedMonthlyIncome": 12000000,
  "slikStatus": "CLEAR",
  "totalOutstanding": 5000000,
  "npwpStatus": "VALID",
  "pepStatus": "CLEAR",
  "dttotStatus": "CLEAR",
  "completedAt": "2026-08-26T10:30:00Z",
  "worstCollectibility": 3,
  "overallAssessment": "FAIL"
}
```

7. API 04 — Credit Decision

POST /api/v1/loan-applications/{applicationId}/decision

API ini digunakan oleh Credit Analyst untuk menyimpan keputusan akhir terhadap suatu pengajuan.

Request Attribute

Attribute	Type	Required	Validation	Example
applicationId	String(36)	Y	Existing application	APP-000123
assessmentId	String(36)	Y	Assessment completed	ASM-000123
analystId	String(36)	Y	Valid analyst	CA-00001
decision	Enum	Y	APPROVED / REJECTED	APPROVED
approvedLimit	Decimal(18, 2)	Conditional	<= policy limit	20000000
approvedTenor	Integer	Conditional	<= product limit	12
interestRate	Decimal(7,4)	Conditional	Product range	12.5000
decisionReasonCode	String(20)	Y	Valid decision code	CREDIT_PASS
decisionNote	String(500)	N	Max 500 chars	Meets credit policy
decisionAt	DateTime	Y	Server timestamp	2026-08-26T11:00:00Z

Business Rule

- Jika decision = APPROVED, maka approvedLimit, approvedTenor, dan interestRate wajib diisi (mandatory).
- Jika decision = REJECTED, maka decisionReasonCode dan decisionNote wajib tersedia.

Contoh Response:

04 - Credit Decision — Success

```
{
  "applicationId": "{{applicationId}}",
  "status": "APPROVED",
  "approvedLimit": 20000000,
  "approvedTenor": 12,
  "interestRate": 12.5,
  "decisionReference": "DEC-000123"
}
```

04 - Credit Decision — Rejected


```
{
  "applicationId": "{{applicationId}}",
  "status": "APPROVED",
  "approvedLimit": 20000000,
  "approvedTenor": 12,
  "interestRate": 12.5,
  "decisionReference": "DEC-000123"
}
```


04 - Credit Decision — Validation Error - Missing Approved Fields


```
{
  "success": false,
  "error": {
    "code": "LOAN-VAL-002",
    "message": "approvedLimit, approvedTenor and interestRate are mandatory when decision =
        APPROVED",
    "details": {
      "field": "approvedLimit",
      "expected": "not null",
      "received": null
    },
    "correlationId": "{{correlationId}}",
    "timestamp": "2026-08-26T11:15:00Z"
  }
}
```


8. API 05 — Loan Booking

POST /api/v1/loans

Purpose

Membuat loan account di Core Banking berdasarkan application yang sudah berstatus APPROVED.

Request Attribute

Attribute	Type	Required	Validation	Example
applicationId	String(36)	Y	APPROVED application	APP-000123
customerId	String(36)	Y	Existing customer	CUS-000123
loanProductCode	String(20)	Y	Valid product	PERSONAL01
loanAmount	Decimal(18,2)	Y	<= approvedLimit	15000000
tenor	Integer	Y	<= approvedTenor	12
interestRate	Decimal(7,4)	Y	Approved rate	12.5000
currency	String(3)	Y	ISO currency	IDR

Attribute	Type	Required	Validation	Example
disbursementAccount	String(30)	Y	Valid account	1234567890
idempotencyKey	String(100)	Y	Unique	BOOK-APP-000123

Critical Business Validation

Booking hanya diproses jika seluruh kondisi berikut terpenuhi secara bersamaan; jika salah satu tidak terpenuhi maka booking ditolak.

```
loanAmount <= approvedLimit
 AND tenor <= approvedTenor
 AND applicationStatus = APPROVED
 AND KYC = VERIFIED
 AND assessment = PASS
```

Transformasi SOA → Core Banking

SOA melakukan transformasi request dari Application Service menjadi format yang dikenali oleh Core Banking System (CBS).

CBS Request

Attribute	Type	Required
customerNumber	String(30)	Y
applicationReference	String(50)	Y
productCode	String(20)	Y
principalAmount	Decimal(18,2)	Y
tenor	Integer	Y
interestRate	Decimal(7,4)	Y
currency	String(3)	Y
disbursementAccount	String(30)	Y
transactionDate	Date	Y
channel	String(20)	Y
correlationId	UUID	Y

CBS Response

Attribute	Type	Example
loanAccountNumber	String(30)	889900123456
loanId	String(36)	LN-000123
bookingStatus	Enum	SUCCESS
bookingReference	String(50)	CBS-BOOK-123
bookingDate	DateTime	2026-08-26T11:10:00Z
errorCode	String(20)	null
errorMessage	String(255)	null

Example Response:

05 - Loan Booking — Success


```
{
  "loanAccountNumber": "889900123456",
  "loanId": "LN-000123",
  "bookingStatus": "SUCCESS",
  "bookingReference": "CBS-BOOK-123",
  "bookingDate": "2026-08-26T11:10:00Z",
  "errorCode": null,
  "errorMessage": null
}
```


05 - Loan Booking — Exceeds Approved Limit


```
{
  "success": false,
  "error": {
    "code": "LOAN-VAL-001",
    "message": "Requested amount exceeds approved limit",
    "details": {
      "field": "loanAmount",
      "expected": "<= 20000000",
      "received": 25000000
    },
    "correlationId": "{{correlationId}}",
    "timestamp": "2026-08-26T11:15:00Z"
  }
}
```


05 - Loan Booking — CBS Booking Failed


```
{
  "loanAccountNumber": null,
  "loanId": null,
  "bookingStatus": "FAILED",
  "bookingReference": null,
  "bookingDate": "2026-08-26T11:10:00Z",
  "errorCode": "CBS-500",
  "errorMessage": "Core Banking service unavailable"
}
```

9. API 06 — Disbursement

POST /api/v1/disbursements

Request Attribute

Attribute	Type	Required	Validation	Example
loanId	String(36)	Y	Active/ready loan	LN-000123
applicationId	String(36)	Y	Existing application	APP-000123
loanAccountNumber	String(30)	Y	CBS account	889900123456
disbursementAmount	Decimal(18, 2)	Y	> 0	15000000
bankCode	String(10)	Y	Valid bank	014
accountNumber	String(30)	Y	Valid account	1234567890
accountName	String(100)	Y	Match validation	Budi Santoso
idempotencyKey	String(100)	Y	Unique	DISB-LN-000123
requestedAt	DateTime	Y	Server timestamp	2026-08-26T11:15:00Z

Disbursement Validation

Sebelum transaksi dikirim ke CBS, sistem menjalankan validasi berurutan berikut:

```
1. Loan Status = READY_FOR_DISBURSEMENT
2. Account Validation
3. Account Status = ACTIVE
4. Account Name = Customer Name
5. Amount <= Loan Amount
6. Idempotency Check
7. Send to CBS
```

Response Attribute

Attribute	Type	Example
disbursementId	String(36)	DISB-000123
loanId	String(36)	LN-000123
amount	Decimal(18,2)	15000000
status	Enum	PROCESSING
transactionReference	String(50)	TRX-000123
requestedAt	DateTime	2026-08-26T11:15:00Z
estimatedCompletionTime	DateTime	2026-08-26T11:16:00Z

Example Response:

06 - Disbursement — Success


```
{
  "disbursementId": "DISB-000123",
  "loanId": "{{loanId}}",
  "amount": 15000000,
  "status": "PROCESSING",
  "transactionReference": "TRX-000123",
  "requestedAt": "2026-08-26T11:15:00Z",
  "estimatedCompletionTime": "2026-08-26T11:16:00Z"
}
```


06 - Disbursement — Account Name Mismatch


```
{
  "success": false,
  "error": {
    "code": "DISB-VAL-001",
    "message": "Account name does not match customer name",
    "details": {
      "field": "accountName",
      "expected": "Budi Santoso",
      "received": "Budi S."
    },
    "correlationId": "{{correlationId}}",
    "timestamp": "2026-08-26T11:15:00Z"
  }
}
```


10. Disbursement Callback

Karena proses disbursement dapat berjalan secara asynchronous, hasil akhir transaksi dikirimkan melalui callback.

POST /api/v1/disbursements/{disbursementId}/callback

Request Attribute

Attribute	Type	Required	Example
disbursementId	String(36)	Y	DISB-000123
loanId	String(36)	Y	LN-000123
transactionReference	String(50)	Y	TRX-000123
status	Enum	Y	SUCCESS
transactionDate	DateTime	Y	2026-08-26T11:16:30Z
amount	Decimal(18,2)	Y	15000000
failureCode	String(20)	Conditional	null
failureReason	String(255)	Conditional	null

Status Flow

```
PENDING -> PROCESSING -> SUCCESS  ==> LOAN ACTIVE
PENDING -> PROCESSING -> FAILED   ==> RETRY / MANUAL INVESTIGATION
```

Example Request:

07 - Disbursement Callback — Success

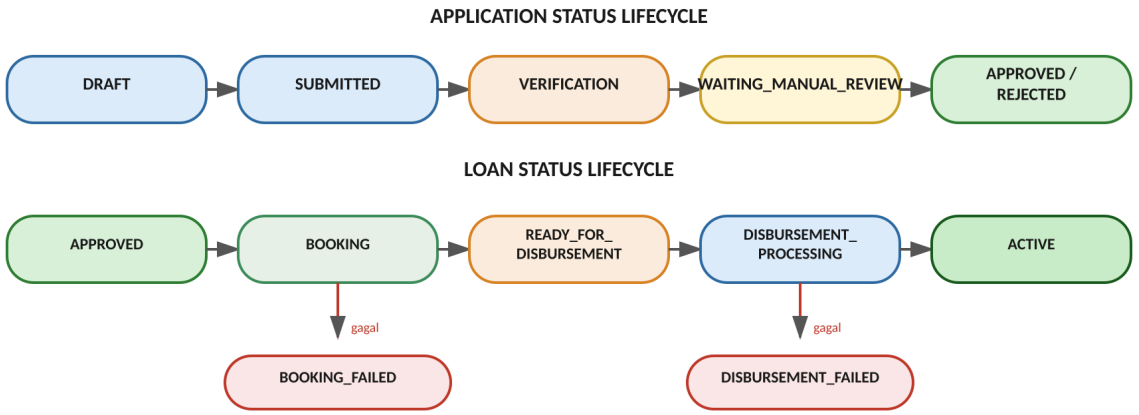
```
{
  "success": true,
  "message": "Callback processed",
  "loanStatus": "ACTIVE"
}
```

07 - Disbursement Callback — Failed Disbursement

```
{
  "disbursementId": "{{disbursementId}}",
  "loanId": "{{loanId}}",
  "transactionReference": "TRX-000124",
  "status": "FAILED",
  "transactionDate": "2026-08-26T11:16:30Z",
  "amount": 15000000,
  "failureCode": "CBS-timeout",
  "failureReason": "Connection timeout to destination bank"
}
```

11. Core Status Transition

Application dan loan masing-masing memiliki state machine tersendiri agar alur transisi status lebih jelas dan terlacak.



Gambar 4. Lifecycle status Application dan Loan

12. Error Response Standard

Seluruh core API menggunakan format error response yang konsisten sebagai berikut.

```
{
 "success": false,
 "error": {
  "code": "LOAN-VAL-001",
  "message": "Requested amount exceeds approved limit",
  "details": {
   "field": "loanAmount",
   "expected": "<= 20000000",
   "received": 25000000
  },
  "correlationId": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-08-26T11:15:00Z"
 }
}
```

Error Category

Code Prefix	Category
AUTH-*	Authentication
KYC-*	KYC
LOAN-VAL-*	Business validation
EXT-*	External API
CBS-*	Core Banking
DISB-*	Disbursement
SYS-*	System

13. Integration Logging

Setiap core API wajib menghasilkan log minimal dengan atribut berikut, yang dapat dibaca dan ditelusuri melalui Kibana berdasarkan correlationId.

Attribute	Type	Example
timestamp	DateTime	2026-08-26T11:15:00Z
serviceName	String	loan-service
endpoint	String	/api/v1/loans
method	String	POST
correlationId	UUID	CORR-123
requestId	UUID	REQ-123
applicationId	String	APP-123
loanId	String	LN-123
externalReference	String	CBS-123
status	String	SUCCESS
responseCode	Integer	200
responseTimeMs	Integer	420

Attribute	Type	Example
errorCode	String	null

14. API Reliability

Ketentuan berikut berlaku untuk seluruh integrasi dengan external API maupun Core Banking, guna menjaga keandalan dan konsistensi transaksi.

14.1 Timeout

- External API timeout: 5 detik
- Core Banking (CBS) timeout: 10 detik

14.2 Retry

Retry hanya dilakukan untuk transient error, yaitu HTTP 502, 503, 504, dan connection timeout, dengan maksimum 3 kali percobaan.

14.3 Idempotency

Idempotency key wajib digunakan pada Loan Booking dan Disbursement karena kedua transaksi tersebut memiliki risiko duplicate financial transaction.

15. Technical Responsibility per Layer

Layer	Responsibility
Mobile App	Input data, upload document, display status
Application Service	Business flow, API exposure, data persistence
SOA	Routing, orchestration, transformation, external integration
External APIs	Identity, employment, credit & compliance verification
Credit Analyst	Manual credit decision & limit
Core Banking	Loan account, booking, financial transaction
Kibana	Log, monitoring, tracing
Jenkins	Build, test, deployment